

This homework has 9 questions, for a total of 100 points and 10 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

(0 pts) 0. **Before you start; before you submit.**

- (a) Before starting this assignment, carefully read [Handout 1](#) (Running Times and Asymptotics) and the “Good Writing” section of [Handout 2](#) (the induction section is optional but recommended), and apply them to your work.
- (b) If applicable, state the name(s) and username(s) of your collaborator(s).

Solution:

1. **L^AT_EX (optional extra credit).**

We (along with the rest of the math-writing world) use L^AT_EX to write all homeworks and exams. Throughout the semester, we strongly encourage you to write all of your homework solutions in L^AT_EX too. This extra credit problem will give you some practice with L^AT_EX.

To compile L^AT_EX code, you can use the website overleaf.com, or download a L^AT_EX editor of your choice. To compile each of the EECS 376 homework assignments in L^AT_EX you will also need to upload the provided `header.tex` file into your project.

L^AT_EX is *not* a “what you see is what you get” word-processing software like you may be accustomed to (Word, Google Docs, etc.). Instead, a L^AT_EX document is a *program* that describes the desired output, and is *compiled* into a human-readable format.

Here are some good resources about L^AT_EX practices:

- [Writing Beautifully in L^AT_EX](#),
- [Short Math Guide for L^AT_EX](#),
- [DOs and DON'Ts when typesetting a document](#).

To receive the extra credit points, you must typeset this **entire** assignment in L^AT_EX, and typeset the two snippets of text given in parts (a) and (b) in L^AT_EX. **Put your solution to this problem in the Verbatim environment as follows so that we can see the L^AT_EX source code in your compiled PDF** (do not use the Verbatim environment for the rest of the assignment):

```
This text is in the \texttt{Verbatim} environment. \emph{This} word is
italized. Here is some math:  $\$1+1=2\$$ .
```

Here are some examples of L^AT_EX syntax you may need:

- In-line math
- Less-than-or-equal-to symbol
- Quote marks
- Bulleted lists

- Display math
- The algorithmic environment

You may copy and modify the source code of this document (e.g. part (b) asks you to write an algorithm in L^AT_EX, and other problems in this document already contain algorithms). You may also use the Internet to search for L^AT_EX syntax.

(5 EC pts)

- (a) The following snippet is about some key features of the big-O notation extracted from [Handout 1](#).

For (non-negative) functions $f(n)$ and $g(n)$, we say “ $f(n) = O(g(n))$ ” if there exist positive constants c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. The key features here are:

- “there exists a positive constant $c \dots f(n) \leq c \cdot g(n)$ ”: this says that $f(n)$ is *upper bounded* by some *constant multiple* of $g(n)$. That is, O -notation “hides,” or ignores, constant factors. Note also that it captures only an *upper bound*: $f(n)$ might actually be *much smaller* than $c \cdot g(n)$, or not. A big-O bound by itself says nothing about which is the case.
- “there exists a positive constant $n_0 \dots$ for all $n \geq n_0$ ”: this says that the upper bound holds for all n *above some constant “threshold.”* However, it says nothing about the relationship between $f(n)$ and $g(n)$ *below* the threshold; it could be that $f(n)$ greatly exceeds (even a huge multiple of) $g(n)$ in that range. Moreover, the threshold n_0 can be *any constant*—it could be one, or ten, or a billion, or a googolplex—and the notation hides its exact value.

Solution:

For (non-negative) functions $f(n)$ and $g(n)$, we say “ $f(n) = O(g(n))$ ” if there exist positive constants c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

The key features here are:

`\begin{itemize}`

`\item “there exists a positive constant  $c$   $\dots$   $f(n) \leq c \cdot g(n)$ ”: this says that  $f(n)$  is upper bounded by some constant multiple of  $g(n)$ .`

That is, O -notation “hides,” or ignores, constant factors.

Note also that it captures only an *upper bound*: $f(n)$ might actually be *much smaller* than $c \cdot g(n)$, or not.

A big-O bound by itself says nothing about which is the case.

`\item “there exists a positive constant  $n_0$   $\dots$  for all  $n \geq n_0$ ”: this says that the upper bound holds for all  $n$  above some constant “threshold.”`

However, it says nothing about the relationship between $f(n)$ and $g(n)$ *below* the threshold; it could be that $f(n)$ greatly exceeds (even a huge multiple of) $g(n)$ in that range.

Moreover, the threshold n_0 can be *any constant*---it could be one, or ten, or a billion, or a googolplex---and the notation hides its exact value.

`\end{itemize}`

Grading notes: Some students have not used the *Verbatim* environment, instead they just wrote the latex code directly. If this is the case, give them a partial credit of 2 points (the question clearly mentions that you have to use the *Verbatim* environment. Make sure that they have not cheated and inserted an image, you can check this by trying to select some portion of the solution.)

+2 Used the `itemize` environment properly.

+2 Used the math symbols correctly in most places exempting few mistakes and penalize one point if “many” (more than 5) mistakes.

+1 Used *emph* or *textit* command where required.

(5 EC pts)

(b) The following snippet is about [Fibonacci numbers](#).

The Fibonacci numbers may be defined by the recurrence relation

$$F_n = F_{n-1} + F_{n-2}$$

with base cases $F_0 = 0$ and $F_1 = 1$. We can compute the n -th Fibonacci number using the following recursive algorithm:

Input: a natural number n
Output: the n -th Fibonacci number

```

1: function FIB( $n$ )
2:   if  $n = 0$  then
3:     return 0
4:   if  $n = 1$  then
5:     return 1
6:   return FIB( $n - 1$ ) + FIB( $n - 2$ )

```

Solution:

`\newpage` The Fibonacci numbers may be defined by the recurrence relation

`\[`

$$F_n = F_{n-1} + F_{n-2}$$

`\]`

with base cases $F_0 = 0$ and $F_1 = 1$. We can compute the n -th Fibonacci number using the following recursive algorithm:

```

\begin{minipage}{\linewidth}
\begin{algorithm}[H]
  \begin{algorithmic}[1]

```

```

\Require{a natural number $n$}
\Ensure{the $n$-th Fibonacci number}
\Function{\text{Fib}}{$n$}
\If{$n = 0$}
  \State \Return 0
\EndIf
\If{$n = 1$}
  \State \Return 1
\EndIf
\State \Return \Call{Fib}{$n-1$} + \Call{Fib}{$n-2$}
\EndFunction
\end{algorithmic}
\end{algorithm}
\end{minipage}

```

Grading notes: Some students have not used the *Verbatim* environment, instead they just wrote the latex code directly. If this is the case, give them a partial credit of 2 points (the question clearly mentions that you have to use the *Verbatim* environment. Make sure that they have not cheated and inserted an image, you can do this by trying to select some portion of the solution. The word “Fibonacci” was misspelled in the problem statement, so don’t take off points if students misspell it.)

- +1 Used the *algorithmic* environment.
- +1 Used *require* and *ensure*.
- +1 Used *Function* command.
- +2 *If* block is properly formatted.

2. Properties of logarithms.

The objective of this problem is to recall some important properties of logarithms that we’ll use throughout the course. First recall the following properties of exponentiation for any real numbers b, x, y , assuming that $b \neq 0$:

P1: $(b^x)^y = b^{xy}$

P2: $b^x = b^y$ if and only if $x = y$, additionally assuming that $b \neq 1$.

Now, for positive real numbers $b \neq 1$ and x , the definition of the base- b logarithm is:

D1: $y = \log_b x$ is the unique real number for which $b^y = x$. That is, it satisfies the identity $b^{\log_b x} = x$.

Combining D1 with P2, we see that $x = y$ if and only if $\log_b x = \log_b y$ (assuming that b, x, y are positive and $b \neq 1$). So, equality is preserved by taking logarithms or exponentials (with the same base) on both sides of an equation, assuming the hypotheses are met. You may use this fact in your solutions.

Prove each of the following properties of logarithms, for any positive real numbers (not necessarily integers) a, b, c, n where $b, c \neq 1$. Cite definition D1 and properties P1 and P2 where you use them; you may also use (with citation) the results of previous parts of the problem. Do not use any other assumptions about, or properties of, logarithms and exponentials.

For reference, we have provided a solution to the first part, which you can use as a model for later solutions.

- (0 pts) (a) $\log_b(a^n) = n \log_b a$. (Be careful: n is not necessarily an integer)

Solution:

By P2, the statement we want to prove is equivalent to $b^{\log_b(a^n)} = b^{n \log_b a}$ (since $b \neq 1$). By D1, the left-hand side is equal to a^n , so it suffices to prove that $a^n = b^{n \log_b a}$. Indeed,

$$b^{n \log_b a} = (b^{\log_b a})^n = a^n,$$

where the first equality is by P1, and the second equality is by D1.

(As a remark, this proof does not rely on the assumption that n is positive, so the statement holds even more generally for any real number n . However, later parts require n to be positive.)

- (3 pts) (b) $\log_b a = \frac{\log_c a}{\log_c b}$. (This is sometimes called the “change-of-base theorem,” because it lets us change the base of the logarithm by dividing by an appropriate factor. Note that $\log_c b \neq 0$ because $b \neq 1$, so the division is well defined.)

Solution: By D1, we have that $a = b^{\log_b a}$. Taking the base- c logarithm of both sides and applying part (a) above, we get that

$$\log_c a = \log_c(b^{\log_b a}) = (\log_b a) \cdot (\log_c b).$$

Dividing both sides by $\log_c b$ yields the desired equality.

Grading notes:

+2 Correctly proves statement

+1 Provides correct justification and steps to achieve proof

- (3 pts) (c) $a^{\log_b n} = n^{\log_b a}$. (This is sometimes called the “log-switching theorem” since it says that we can “switch” a and n .)

Solution: Taking the base- b logarithm of both sides, the statement we want to prove is equivalent to $\log_b(a^{\log_b n}) = \log_b(n^{\log_b a})$. This is true because by part (a), the left side is equal to $\log_b n \cdot \log_b a$ and the right side is equal to $\log_b a \cdot \log_b n$.

Grading notes:

+2 Correctly proves statement

+1 Provides correct justification and steps to achieve proof

- (3 pts) (d) The ratio $\frac{\log_b n}{\log_c n}$ (for $n \neq 1$) depends only on b and c , not n .

This implies that we can often omit the (constant) base when writing logarithms in asymptotic notation, e.g., we can write $O(\log n)$ instead of $O(\log_2 n)$, because $O(\cdot)$ hides constant factors that do not depend on n . However, we cannot omit the base when the logarithm appears in the exponent, as in $O(3^{\log_2 n})$.

Solution: By applying part (b) twice, we get that

$$\frac{\log_b n}{\log_c n} = \frac{\log_2 n / \log_2 b}{\log_2 n / \log_2 c} = \frac{\log_2 c}{\log_2 b},$$

which depends only on b and c , as desired.

Grading notes:

+2 Correctly proves statement

+1 Provides correct justification and steps to achieve proof

3. Welcome to EECS 376!

To express our excitement for the start of EECS 376, we present an algorithm that prints "Welcome to EECS 376!" multiple times, based on input parameter n .

Input: positive integer n represented in binary

```
1: function WELCOME( $n$ )
2:   for  $i = 1, 2, \dots, n$  do
3:     for  $j = 1, 2, \dots, i$  do
4:       PRINT("Welcome to EECS 376!")
```

- (4 pts) (a) Give the *tightest correct asymptotic* (big- O) bound, as a function of n , on the worst-case running time of the WELCOME(n) algorithm.

Solution: The print function is called i times during each iteration of the outer loop. Thus, the print function is called a total of $\sum_{i=1}^n i = (n+1)n/2 = O(n^2)$ times. Each call to the print function takes $O(1)$ (constant) time, so the worst-case running time is $O(n^2)$.

Grading notes:

+2 For getting the correct asymptotic bound.

+2 For giving an explanation of $O(n^2)$ by either mentioning number of prints as the closed form $n(n+1)/2$ or the summation $\sum_{i=1}^n i$.

- (4 pts) (b) Let s be the input size of the integer n , i.e., the number of bits needed to represent n in binary. Give the tightest correct asymptotic (big- O) bound, as a function of s , on the worst-case running time of the WELCOME algorithm. State whether or not the algorithm is *efficient*, i.e., runs in at most polynomial time with respect to the input size s .

Solution: This is **not** polynomial—it is exponential—in the input size, so it is not efficient.

The running time is $O(n^2)$, but the input size is $s = \lceil \log_2(n+1) \rceil$. So, $n = \Theta(2^s)$.

This means that if we write $O(n^2)$ in terms of s , we get $O(2^{2s})$, which is exponential in terms of the input size.

Grading notes:

+2 For obtaining the correct runtime bound of $O(4^s)$.

+2 For stating that it is not efficient, with some justification.

- (4 pts) (c) Does your answer in [Part b](#) change if n is represented in decimal (base-10) instead? That is, let s' be the input size of the integer n , i.e., the number of digits needed to represent n in base-10. Give the tightest correct asymptotic (big- O) bound, as a function of s' , on the worst-case running time of the WELCOME algorithm. State whether or not the algorithm is *efficient*, i.e., runs in at most polynomial time with respect to the input size s' .

Solution: This is again not polynomial. We can use the same logic as in [Part b](#). The running time is $O(n^2)$, but the input size is $s' = \lceil \log_{10}(n+1) \rceil$. So, $n = \Theta(10^{s'})$.

This means that if we write $O(n^2)$ in terms of s' , we get $O(10^{2s'})$, which is exponential in terms of the input size.

Takeaway: More generally, if the input integer n is represented in any base (except base-1), then the base will not affect whether the running time is polynomial versus exponential.

Grading notes:

+2 For obtaining correct runtime bound $O(100^{s'})$.

+2 For stating that it is not efficient, with some justification.

- (4 pts) (d) Now consider a different algorithm $\mathcal{A}$ that takes an input an integer n and has worst-case running time $O(\log n)$.

Suppose n is represented in binary and let s be the input size (number of bits in n). Give the tightest correct asymptotic (big- O) bound, as a function of s , on the worst-case running time of algorithm $\mathcal{A}$. State whether or not the algorithm is *efficient*.

Does your answer change if n is represented in base-10 instead? That is, let s' be the input size (number of digits needed to represent n in base-10). Give the tightest correct asymptotic (big- O) bound, as a function of s' , on the worst-case running time of algorithm $\mathcal{A}$. State whether or not the algorithm is *efficient*.

Solution:

As in the previous parts, $s = \lceil \log_2(n+1) \rceil$ so $n = \Theta(2^s)$. So, the running time of $O(\log n)$ can be expressed as $O(\log 2^s) = O(s)$, by [Part a](#) of Problem 2. This is efficient.

Similarly, $s' = \lceil \log_{10}(n+1) \rceil$, so $n = \Theta(10^{s'})$. So, the running time of $O(\log n)$ can be expressed as $O(\log 10^{s'}) = O(s')$. This is efficient. Note that the dependence on s versus s' in the running time is exactly the same. They are both linear.

Takeaway: More generally, if the input integer n is represented in any base (except base-1) and the algorithm runs in polynomial time, then the base will not affect the asymptotic running time *at all*.

Grading notes:

+2 For obtaining correct runtime bound $O(s)$ and $O(s')$.

+2 For stating that it is efficient, with some justification.

4. Practice with asymptotics (“big-Oh, big-Omega, big-Theta”).

Let $f(n)$ and $g(n)$ be positive functions. Recall that $f(n) = O(g(n))$ if there exist constants $c, n_0 > 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

- In words, this says: “ $f(n)$ is *at most* (or: is *upper-bounded by*) a *constant multiple* of $g(n)$, for *all large enough* n .” (“All large enough n ” refers to the clause “all $n \geq n_0$ ”, i.e., all n beyond some fixed threshold.)
- It can be shown that:
 1. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is finite, $f(n) = O(g(n))$ is true.
 2. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is infinite, $f(n) = O(g(n))$ is false.
 3. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ does not exist, we cannot conclude whether $f(n) = O(g(n))$ is true or false. For example, for $f(n) = |\sin(n)|$ and $g(n) = 2$ the limit does not exist, but $f(n) = O(g(n))$.

Similarly, $f(n) = \Omega(g(n))$ if there exist constants $c, n_0 > 0$ such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.

- In words, this says that “ $f(n)$ is *at least* (or: is *lower-bounded by*) a positive-constant multiple of $g(n)$, for all large enough n .”
- It can be shown that:
 1. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is greater than zero (including infinity), $f(n) = \Omega(g(n))$ is true.
 2. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is zero, $f(n) = \Omega(g(n))$ is false.
 3. If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ does not exist, we cannot conclude whether $f(n) = \Omega(g(n))$ is true or false.

Moreover, it can be seen that $f(n) = \Omega(g(n))$ is equivalent to $g(n) = O(f(n))$, since $f(n) \geq c \cdot g(n)$ if and only if $g(n) \leq (1/c) \cdot f(n)$, and c is a positive constant if and only if $1/c$ is a positive constant.

Finally, $f(n) = \Theta(g(n))$ if both $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

For the following pairs of functions, state, with rigorous proof, whether or not each of the following hold: $f(n) = O(g(n))$, $f(n) = \Omega(g(n))$, $f(n) = \Theta(g(n))$.

Hint: You might find L'Hôpital's Rule useful, which says (in particular): if $f(n), g(n)$ are differentiable, $g'(n) \neq 0$ for all large enough n , and $\lim_{n \rightarrow \infty} g(n) = \infty$, then $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$ whenever the latter limit exists. (Here f' and g' are respectively the derivatives of f and g .) You may use a calculator or computer to calculate derivatives.

- (3 pts) (a) $f(n) = \log_2(n^3)$, $g(n) = \log_5(n^{1/8})$.

Optional ungraded challenge: generalize to similar-looking f and g , beyond specific constant bases and exponents.

Solution: All three conditions hold, i.e., $f(n) = \Theta(g(n))$. By the previous problem on logarithms, $f(n) = 3 \log_2 n$ and

$$g(n) = \frac{\log_5 n}{8} = \frac{\log_2 n}{8 \log_2 5}.$$

So $f(n)/g(n)$ is a positive constant, so its limit as n approaches infinity is the same constant, and hence $f(n) = \Theta(g(n))$.

More generally, if $f(n), g(n)$ are logarithms, with *any* (not necessarily the same) constant bases greater than one, of *any* polynomials in n (not necessarily of the same degree), then $f(n) = \Theta(g(n))$. Try to prove this!

Grading notes:

- +1 For obtaining correct conclusion about $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$
- +1 For obtaining correct conclusion about $f(n) = \Theta(g(n))$
- +1 For providing correct justification

- (4 pts) (b) $f(n) = (376 + \sin(n)) \cdot n$, $g(n) = 203n$.

Optional ungraded challenge: generalize to similar-looking f and g , beyond specific constant coefficients or exponents.

Solution: Notice that in this case the limit $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ does not exist, since $\sin(n)$ “oscillates” as n grows. So, we instead address the questions by working directly with the definitions of O and Ω .

Since $-1 \leq \sin(n) \leq 1$ for all n , we have $375n \leq f(n) \leq 377n$. (In other words, we have “sandwiched” $f(n)$ between two constant multiples of the same function.) Since $g(n) = 203 \cdot n \leq 375 \cdot n \leq f(n)$ for all n , we have $g(n) = O(f(n))$, or equivalently, $f(n) = \Omega(g(n))$. Similarly, $f(n) \leq 377 \cdot n \leq 2 \cdot g(n)$, hence $f(n) = O(g(n))$. Thus, $f(n) = \Theta(g(n))$.

More generally, if $f(n)$ is both upper- and lower-bounded (“sandwiched”) by constant multiples of the same function $h(n)$ (for all large enough n), then any asymptotic bounds that apply to $h(n)$ also apply to $f(n)$. Another way to say this is that asymptotic bounds are *transitive*: if $f(n) = O(h(n))$ and $h(n) = O(g(n))$, then $f(n) = O(g(n))$; and similarly for Ω . (Try to prove this!)

Above we used the fact that $f(n) \leq 377n$ hence $f(n) = O(n)$, and $n = O(g(n))$, so $f(n) = O(g(n))$. (The Ω bounds follow similarly from $f(n) \geq 375n$.)

Grading notes:

+1 For obtaining correct conclusion about $f(n) = O(g(n))$

+1 For obtaining correct conclusion about $f(n) = \Omega(g(n))$

+1 For obtaining correct conclusion about $f(n) = \Theta(g(n))$

+1 For providing correct justification

(6 pts) (c) $f(n) = n^3 \cdot 2^{3n}$, $g(n) = 3^{2n}$.

Optional ungraded challenge: generalize to similar-looking f and g , beyond specific constants in the bases and exponents.

Solution: Notice that $f(n) = n^3 \cdot 8^n$ and $g(n) = 9^n$; we will use these forms in what follows.

We first show that $f(n) = O(g(n))$. We give a proof directly according to the definition, by giving suitable constants c, n_0 and showing that $n^3 \cdot 8^n \leq c \cdot 9^n$ for all $n \geq n_0$. Letting $c = 1$ for simplicity, what we want to show is equivalent to $n^3 \leq (9/8)^n$ for all $n \geq n_0$. Using a calculator, we see that this inequality holds at $n = 232$, so we take $n_0 = 232$. Then we observe that beyond this threshold, the right-hand side grows much more quickly than the left-hand side: when n increases by one, the right-hand side grows multiplicatively by a factor of $9/8 = 1.125$, while the left-hand side grows by a factor of *at most* $(233/232)^3 \leq 1.013$. So, the inequality continues to hold for all $n \geq n_0$, as needed.

Now we show that $f(n) \neq \Omega(g(n))$. We can prove this without much difficulty using limits. Note that in $f(n)/g(n)$, we can move factors between the numerator and denominator as is convenient, to get nicer derivatives when applying L'Hôpital's Rule. In particular,

$$\frac{f(n)}{g(n)} = \frac{n^3 \cdot 8^n}{9^n} = \frac{n^3}{(9/8)^n}$$

so by repeated applications of L'Hôpital's Rule,

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} &= \lim_{n \rightarrow \infty} \frac{n^3}{(9/8)^n} = \lim_{n \rightarrow \infty} \frac{3n^2}{(9/8)^n \ln(9/8)} = \lim_{n \rightarrow \infty} \frac{6n}{(9/8)^n \ln^2(9/8)} \\ &= \lim_{n \rightarrow \infty} \frac{6}{(9/8)^n \ln^3(9/8)} = 0, \end{aligned}$$

where the last equality holds because the numerator is constant and the denominator increases without bound as n grows. Since the limit exists and is zero, we have that $f(n) \neq \Omega(g(n))$.

Alternatively, we can prove that $f(n) \neq \Omega(g(n))$ directly according to the definition. We need to prove the logical negation of the statement “ $n^3 \cdot 8^n = \Omega(9^n)$,” which is: *for all* positive constants c, n_0 , we have $n^3 \cdot 8^n < c \cdot 9^n$ for *some* $n \geq n_0$. So, letting $c, n_0 > 0$ be some arbitrary constants, we need to find some $n \geq n_0$ (which may depend on c, n_0), for which $n^3 < c \cdot (9/8)^n$. The key intuition is that the exponential on the right eventually grows arbitrarily larger than the polynomial on the left, so any large enough n does indeed suffice.

To justify this, with the help of a computer algebra system like Mathematica, we can determine that the inequality $n^3 < c \cdot (9/8)^n$ holds whenever n is greater than some complicated-looking expression we call K_c (which involves various logarithms and esoteric-sounding “product log functions”). The exact form of K_c does not really matter; the key point is that K_c *depends only on* c , and hence is a constant because c is a constant. Therefore, by taking any $n > \max(n_0, K_c)$, we do indeed satisfy the inequality for some $n \geq n_0$, as needed.

More generally, any exponential function in n with base greater than 1 grows asymptotically faster than any polynomial-bounded function in n . That is, $n^c = O(a^n)$ and $n^c \neq \Omega(a^n)$ for any constants $a > 1, c$. (Try to prove this!) Although $f(n)$ is not polynomial in the above question, by dividing both $f(n)$ and $g(n)$ by 8^n we can put them in the desired form.

Grading notes:

- +2 For obtaining correct conclusion about $f(n) = O(g(n))$
- +1 For obtaining correct conclusion about $f(n) = \Omega(g(n))$
- +1 For obtaining correct conclusion about $f(n) = \Theta(g(n))$
- +2 For providing correct justification

5. Prof. Upsilon and their oopsie claims.

Prof. Υ is an absent-minded computer scientist, known for their occasional “oopsie” moments. Recently, Prof. Υ came up with three algorithms with the following running times:

- Algorithm X has worst-case running time $T_X(n) = O(n^2)$
- Algorithm Y has worst-case running time $T_Y(n) = \Theta(n \log n)$
- Algorithm Z has worst-case running time $T_Z(n) = \Theta(n)$

Recall that *worst-case* running time of an Algorithm A means the *maximum* number of steps for which A runs, over all possible inputs of size n .

Now, Prof. Υ has made a series of claims about these algorithms. State, with proof, whether Prof. Υ ’s claim is necessarily true, necessarily false, or not necessarily either based on the given information. If it is not necessarily either, give an example when it is true and an example when it is false. For instance, such an example could be of the form: “Suppose Algorithm Y takes time $8n \log n$ for some inputs and time $4n$ for the remaining inputs.”

Solution: Before considering the specific claims, we highlight the main conceptual points for this problem:

- An algorithm can take varying amounts of time on different inputs, even ones of the same size. (For example, consider insertion sort, which is fast when its input array is already sorted, but much slower when its input is far from sorted.) In this class, we care mainly about the *worst-case* running time over all inputs of a particular size, so that we have a runtime guarantee no matter what the input is. But on certain inputs, an algorithm might perform better than its worst-case behavior.
- An $O(\cdot)$ bound is merely an (asymptotic) *upper bound*. A function that grows much more slowly than $g(n)$ is still considered to be $O(g(n))$. Similarly, an $\Omega(\cdot)$ bound is merely an (asymptotic) *lower bound*; a function that grows much more quickly than $g(n)$ is still considered to be $\Omega(g(n))$. By contrast, a $\Theta(\cdot)$ bound is both an (asymptotic) upper *and* lower bound. Any function that is $\Theta(g(n))$ must be ‘sandwiched’ between two constant multiples of $g(n)$ (above a certain threshold for n), i.e., it must scale just as $g(n)$ does, as n grows large.
- Asymptotic bounds like $O(\cdot)$ and $\Omega(\cdot)$ say how a function (often, an algorithm’s runtime) scales *as the input size n grows large*, beyond some threshold (denoted n_0). They do not say anything about the function’s behavior for “small” values of n (below the threshold). Without any additional hypotheses, the function’s behavior below the threshold can be arbitrary.

(4 pts) (a) “Algorithm Y has running time $O(n^2)$.”

Solution:

This is necessarily true because $T_Y(n) = \Theta(n \log n) \implies T_Y(n) = \Omega(n \log n)$ and $O(n \log n)$. Since $n \log n = O(n^2)$, we can indeed upper bound the runtime of Y by n^2 .

Grading notes: There are multiple valid solutions here, as it is possible to come with multiple examples here, so make sure to go over them carefully.

+2 For figuring out that runtime of Y is $O(n \log n)$.

+2 For figuring out that $n \log n$ can be upper bounded by n^2 .

(6 pts) (b) “On every sufficiently large input, Algorithm Z runs faster than Algorithm Y .”

Solution: The claim is **not necessarily true**. $T_Y(n)$ refers to Y ’s *maximum* running time over all inputs of size n ; it says nothing about how Y performs on non-worst-case inputs. So, even if we know that $T_Z(n) < T_Y(n)$ for sufficiently large n , we still cannot conclude that Z is faster than Y on *every* size- n input. To specify an example, Y could run in time $n \log n$ for some inputs and time $\sqrt{n}$ for the remaining inputs, while Z could run in time n for all inputs.

The claim is **not necessarily false**; it can be true because on *all* inputs of size n , Z might run in time exactly n , and Y might run in time exactly $n \log_2 n$, meaning that Z runs faster than Y on every input.

Grading notes:

- +1 Show examples of running times of Y, Z for which statement is true.
- +2 Prove that for these Y, Z , the statement is indeed true.
- +1 Show examples of running times of Y, Z for which statement is false.
- +2 Prove that for these Y, Z , the statement is indeed false.

- (6 pts) (c) “For all large enough n , there is an input of size n for which Y runs faster than X .”

Solution: The claim is **not necessarily true**; it can be false because the $O(n^2)$ upper bound on $T_X(n)$ could be very *loose*, and X might actually run in much faster than quadratic time (for all inputs). For example, it could be the case that $T_X(n) = 10$ for all inputs, whereas Y takes time $100n \log_2(2n) = \Theta(n \log n)$ for *every* input of size n . Then because $100n \log_2(2n) \geq 100n > 10$ for every $n \geq 1$, X is faster than Y on *every* input.

The claim is **not necessarily false**; it can be true because the $O(n^2)$ upper bound on $T_X(n)$ might actually be tight. For example, it could be that $T_X(n) = n^2$ and $T_Y(n) = n \log_2 n$. Then for each value of n , there is some *worst-case* input x_n for X , i.e., X run on x_n takes time exactly n^2 . For this input, Y takes time *at most* $n \log_2 n$. Because $n \log_2 n < n^2$ for all $n \geq 1$, Y is faster than X on all the inputs x_n .

Important subtlety: For the previous argument, we need to consider *worst-case* size- n inputs for X . (An alternative way to do this is by adopting an example where X takes time $T_X(n)$ for *every* size- n input; this makes every input a worst-case one.) This is for the same reason discussed previously: we need a lower bound on X ’s runtime for certain inputs, and $T_X(n)$ is merely the *worst-case* runtime of X on size- n inputs; X might run much faster on non-worst-case inputs.

Grading notes: There are multiple valid solutions here, as it is possible to come with multiple examples here, so make sure to go over them carefully.

- +1 Show examples of running times of X, Y for which statement is true.
- +2 Prove that for these X, Y , the statement is indeed true.
- +1 Show examples of running times of X, Y for which statement is false.
- +2 Prove that for these X, Y , the statement is indeed false.

- (6 pts) (d) “For all large enough n , there is an input of size n for which Z runs faster than Y .”

Solution: This is necessarily true. For each value of n , there is some *worst-case* size- n input y_n for Y , i.e., Y run on y_n takes time exactly $T_Y(n)$. Then because $T_Y(n) = \Omega(n \log n)$, there is a positive constant $c > 0$ such that for all large enough n , Y run on y_n takes time *at least* $cn \log_2 n$. Because $T_Z(n) = O(n)$, there is a positive constant $c' > 0$ such that for all large enough n , Z run on y_n takes time *at most* $c'n$. Finally, because $c'n < cn \log_2 n$ for all large enough n (namely, when $n > 2^{c'/c}$), we conclude that Z runs faster than Y on y_n for all large enough n .

Important subtlety: For the previous argument, we need consider *worst-case* size- n inputs to Y , not arbitrary ones. This is because we need a lower bound on Y 's runtime for certain inputs. As discussed in the previous parts, $T_Y(n)$ represents Y 's runtime on worst-case size- n inputs, and Y could be much faster on non-worst-case inputs.

As a remark, it is important to carefully understand what the claim does *not* say:

- It does not say that Z is faster than Y on *all* inputs, or even those of large enough size. Indeed, it could be the case that Z is *slower* than Y on almost all inputs—say, more than 99.9% of them!

Again, $T_Y(n)$ and $T_Z(n)$ only refer to the *worst-case* running times of Y and Z on size- n inputs, not the “best case” or “typical case.”

- It does not say that for *every* n , there is a size- n input on which Z is faster than Y . Indeed, it could be the case that Z is *slower* than Y for *all* inputs of size up to (say) 2^{1000} , which is vastly more than the number of atoms in the universe!

The asymptotic expressions only say what happens for “large enough” values of n , i.e., as n grows toward infinity; but they do not say what the specific threshold of “large enough” is.

Grading notes:

- +3 Writing down the running times of Y and Z more concretely, in terms of constants hidden by the Θ term.
- +3 For working out a large enough n and explaining why the statement is necessarily true.

6. Set of proofs by contra-

Recall that a *set* is a collection of distinct elements. Below are some important set notations:

- **Elements of a set:** $x \in A$ means x is an element of set A .
- **Complement of a set:**¹ $\overline{A}$ refers to the set of all elements in the universal set U that are not in A .
- **Subset:** $A \subseteq B$ means every element of A is also an element of B .
- **Intersection of sets:** $A \cap B$ is the set of elements that are in both A and B .

¹In some texts, you may encounter notations like A^C , A^c , A' , etc. to refer to the complement of A .

- **Union of sets:** $A \cup B$ is the set of elements that are in A , in B , or in both.
- **Set difference:** $A \setminus B$ is the set of elements that are in A but not in B . We can also write it as $A \cap \overline{B}$.

(4 pts) (a) Recall the transitive property of the subset relation:

If $A \subseteq B$ and $B \subseteq C$, then $A \subseteq C$.

Prove the above property by contradiction.

Solution: Suppose for contradiction that $A \subseteq B$ and $B \subseteq C$ but $A \not\subseteq C$. This means there exists an element $x \in A$ but $x \notin C$. Since $A \subseteq B$, we know that x must be in B . Similarly, since $B \subseteq C$, x must also be in C . This contradicts the assumption that $x \notin C$, so A must be a subset of C .

Grading notes:

- +2 For writing down the correct statement that is contradicting the conclusion in the question.
- +2 For correctly arriving at a contradiction, beginning from the statement written down at the start.

(4 pts) (b) Recall the DeMorgan's Law, which says:

If $x \in \overline{A \cap B}$, then $x \in \overline{A} \cup \overline{B}$.

State the contrapositive of the statement. Then, prove the contrapositive.

Solution: The contrapositive of the statement is: if $x \notin \overline{A} \cup \overline{B}$, then $x \notin \overline{A \cap B}$ (or equivalently, $x \in A \cap B$). If $x \notin \overline{A} \cup \overline{B}$, then $x \notin \overline{A}$ and $x \notin \overline{B}$, which means $x \in A$ and $x \in B$. Therefore, $x \in A \cap B$.

Grading notes:

- +2 For writing down the contrapositive correctly.
- +2 For proving the contrapositive.

7. Requirements for the potential method.

Recall that the potential method requires several properties to hold in order to prove that a given process must eventually halt, within a certain number of “time units.” These requirements include (but are not limited to):

1. the potential cannot go below some fixed lower bound; or if it does, the process must immediately halt.
2. the potential must *decrease by (at least) some fixed positive amount* with each time unit.

For each of the following processes and candidate potential functions, state with justification whether they satisfy each of the above properties. If not, either describe an alternative potential function that does satisfy both properties (and allows us to conclude that the process must halt),

or briefly explain why the process might not ever halt (and hence there is *no* valid potential function for the process).

- (4 pts) (a) You take a road trip from Michigan to California, which are about 2360 miles apart. On each day, you drive *half* of your remaining distance to California, and then stop to see the sights. So, you drive (roughly) 1180 miles on the first day, 590 miles on the second day, 295 miles on the third day, and so forth. Define a time unit to be one day of driving, and the potential s_i after i days of driving to be your remaining distance to California. The road trip ends once you reach California.

Solution:

The given potential function certainly satisfies [Item 1](#): the distance to California cannot be less than zero (and when it reaches zero, the road trip ends). For [Item 2](#), although the potential function does decrease with each time unit (day of driving), it does *not* decrease by a *fixed positive amount* with every time unit: the amount of decrease is cut in half each day, always remaining positive but becoming arbitrarily close to zero. More precisely, on the i th day the potential decreases by only $2360/2^i > 0$ miles, which is less than any fixed constant once i is large enough. For example, on the 31st day the potential decreases by only $2360/2^{31}$ miles, which is less than one-tenth of an inch. Under the usual mathematical interpretation, in which distances are nonnegative real numbers, the road trip never quite reaches California, so it never halts. Hence, there is no valid potential function for this process.

Special case (accepted alternative). We will also accept a physical interpretation in which distance consists of finite number of smallest units (for example, atoms), provided that this assumption is explicitly stated. If the remaining distance is measured as a nonnegative integer number of such units and halving using a suitable rounding convention, then any decrease is by at least one unit, and eventually the remaining distance becomes zero. Under this interpretation, the road trip does halt and the given potential, measured in these units, is a valid potential function.

Notice that this kind of “Zeno’s paradox” phenomenon on the potential values cannot arise if we define them to be integers, as in the special physical interpretation where distance consists of a finite number of smallest units. In this case, any decrease in potential must be a decrease by at least 1 (which is a positive constant), so for [Item 2](#) it suffices to show that the potential function merely decreases with each time unit. (Under the usual interpretation, because this road trip scenario does not terminate, it is not possible to define a valid potential function for it that takes on only integer values.)

Grading notes:

Full credit may be awarded for either the usual real-valued interpretation or an explicitly stated discrete physical interpretation, provided the student’s answers are consistent with that interpretation.

+1 For correctly concluding and proving Item 1 is satisfied

+1 For correctly concluding and proving Item 2 is NOT satisfied in the real-valued model, or that is satisfied in a discrete physical model

- +1 For correctly concluding process does NOT halt in the real-valued model, or that it does halt in a discrete physical model
- +1 For correct justification of the corresponding halting or non-halting conclusion

- (6 pts) (b) You initially have 203 coins lying flat on a table, all showing heads. You repeatedly perform the following action: if it is possible to do so, you *must* choose a pair of coins showing heads and turn over *both* of them to show tails. Otherwise, you may turn over *any one or two coins* of your choice. You continue until no coins are showing heads. Define a time unit to be one “action” that turns over one or two coins, and the potential s_i to be the number of heads showing after i actions.

Solution: This trivially satisfies [Item 1](#) (because there cannot be a negative number of heads), but not [Item 2](#). This is because the initial number of heads is odd, and once there is just one head showing (after necessarily revealing 101 pairs of heads), the next turn might not decrease the number of heads: there is an option to increase the number of heads by one or two (by revealing one or two tails), or to not change the number of heads at all (by revealing one head and one tail).

Although the process *can* halt by making certain choices (e.g., once there is just one head showing, turn it over), it is not *required* to halt. The process can continue forever because it is possible to always keep exactly one head showing, by repeatedly turning over both a head and a tail once there is just one head showing (or more generally, keep an odd number of heads showing). So, there is no valid potential function for this process.

Grading notes:

- +1 For correctly concluding and proving Item 1 is satisfied
- +2 For correctly concluding and proving Item 2 is NOT satisfied
- +1 For correctly concluding process does NOT halt
- +2 For correct justification of not halting

- (8 pts) (c) Consider this process for sorting a given array of n distinct integers $A[1, \dots, n]$ in increasing order: repeatedly select any two out-of-order elements (i.e., two elements $A[j]$ and $A[k]$ where $j < k$ but $A[j] > A[k]$) and swap them in the array. The process continues until there are no out-of-order elements, in which case the array is then in sorted order. Define a time unit to be one swap, and the potential s_i after i swaps to be the number of array elements that are *not* in the positions they would occupy if the array were sorted.

Solution: The proposed s_i does not satisfy [Item 2](#), since a single exchange may not reduce the number of “out of place” elements. For example, consider swapping 4 and 5 in the array $[5, 4, 3, 2, 1]$; in this case, every element except 3 is out of place both before and after the swap.

However, a different potential function *does* allow us to prove that this process must

eventually halt (no matter what pairs of elements we choose to swap). Specifically, we define the new potential to be the **number of out-of-order (OOO) pairs of elements** after the i th swap. This clearly satisfies [Item 1](#), as it cannot go below zero; and indeed, the process clearly ends if the potential becomes zero. Also, the initial potential is at most $\binom{n}{2} = n(n-1)/2$, the number of pairs of distinct elements.

Although it might seem intuitive that this potential should decrease with every swap of two OOO elements, a proof is not obvious. This is because a swap can affect which pairs of elements (and array indices) are OOO, apart from the swapped pair. It could produce some *new* OOO pairs, and also *remove* some other OOO pairs (other than the swapped one). Nevertheless, we prove that each swap must decrease the *total* number of OOO pairs by at least 1, as follows.

Let $j < k$ be the indices of the out-of-order pair that is swapped, so $A[j] > A[k]$ before the swap. We consider the net change in the number of OOO pairs due to this swap. Consider an arbitrary pair of array indices:

- If *both* indices are involved in the swap (i.e., they are j and k), then the swap puts the corresponding OOO elements in order, so this index pair contributes a net -1 to the potential.
- If *neither* index is involved in the swap, then the corresponding elements *remain* either in order, or out of order—i.e., this index pair does not contribute any net change to the potential.

So, it just remains to consider the net change in the potential due to the index pairs involving *exactly one* of the indices j and k . We claim that the net change due to these pairs is at most zero (it may be negative), and hence the potential decreases by at least 1 overall. Consider any index $l \notin \{j, k\}$ and its pairings with j and k . We have three cases:

1. If $l < j$, then the swap has no net effect on the number of OOO pairs among l, j and l, k , because $A[j]$ and $A[k]$ still appear *after* $A[l]$ in the array, before and after the swap.
2. Similarly, if $l > k$, then the swap has no net effect on the number of OOO pairs among j, l or k, l .
3. Finally, if $j < l < k$, then swapping $A[j]$ and $A[k]$ cannot increase the number of OOO pairs among j, l and l, k , because the swap makes $A[j]$ smaller and $A[k]$ larger (and doesn't change $A[l]$). That is, if index pair j, l is OOO *after* the swap, then it must also have been OOO *before* the swap; and similarly for index pair l, k .

Altogether, since we have shown that the potential is initially at most $\binom{n}{2}$, is lower-bounded by zero, and decreases by at least 1 with each step (swap), we can conclude that the process must halt within at most $\binom{n}{2}$ steps (no matter what swaps are used).

Grading notes:

- +1 For correctly concluding and proving Item 1 is satisfied for the given potential function
- +2 For correctly concluding and proving Item 2 is NOT satisfied for the given potential function
- +2 For giving a correct potential function
- +1 For correctly explaining and proving Item 1 is satisfied for a correct potential function
- +2 For correctly explaining and proving Item 2 is satisfied for a correct potential function

8. Euclid's algorithm, extended.

Given two integers x, y (that are not both zero), one may wonder what values can be obtained by summing integer multiples of x and y , i.e., expressed as $ax + by$ for some (not necessarily positive) integers a, b . It is impossible to obtain any positive integer *smaller* than their GCD $g = \gcd(x, y)$, because $ax + by$ must be divisible by g . But, it turns out that the GCD of x and y *can* always be expressed as $ax + by$ for some integers a, b ; this theorem is known as *Bézout's identity* (pronounced "BAY-zoo").

In this problem, you will modify the standard Euclidean algorithm to output not only $g = \gcd(x, y)$ itself, but also a pair (a, b) of integers for which $ax + by = g$. Such integers are called *Bézout coefficients* for x, y . (As we will see later, this is very important for cryptography!) We give most of the algorithm below:

Input: integers $x \geq y \geq 0$, not both zero

Output: a triple (g, a, b) of integers where $g = \gcd(x, y) = ax + by$

```

1: function EXTENDED_EUCLID( $x, y$ )
2:   if  $y = 0$  then
3:     return  $(x, 1, 0)$  // Base case:  $1x + 0y = x = \gcd(x, 0)$ 
4:   else
5:     Divide  $x$  by  $y$ , writing  $x = qy + r$  for integer quotient  $q$  and remainder  $0 \leq r < y$ 
6:      $(g, a', b') \leftarrow \text{EXTENDED\_EUCLID}(y, r)$ 
7:     [FOR YOU TO DETERMINE: compute appropriate  $a, b$ ]
8:     return  $(g, a, b)$ 

```

(6 pts)

- (a) State what a and b should be on Line 7, and prove that that $ax + by = g$.

Hint: by recursion/induction, we know that $g = \gcd(y, r)$ and $a'y + b'r = g$.

Solution:

We can derive good choices for a, b as follows. By induction (i.e., correctness of the recursive call), we know that $a'y + b'r = g$. We can rearrange the equation from line 5

to get $r = x - qy$. We want a and b to satisfy $ax + by = g$, i.e., we want to make

$$ax + by = a'y + b'r = a'y + b'(x - qy) .$$

To do this, we match up the coefficients of x and y (respectively) from the leftmost and rightmost expressions, which suggests that we should set $a = b'$ and $b = a' - b'q$. We can then verify that these values are correct:

$$\begin{aligned} ax + by &= b'x + (a' - b'q)y \\ &= a'y + b'(x - qy) \\ &= a'y + b'r = g , \end{aligned}$$

as required.

Grading notes:

+3 For obtaining the correct expressions for a, b in terms of a', b', q .

+3 For proving/verifying that the expressions of a, b obtained are indeed correct.

(8 pts)

- (b) **Run the Extended Euclid algorithm by hand** to find Bézout coefficients for the input $(x, y) = (376, 203)$, and show that the output is correct. (You may use a calculator/computer only for the division steps.) Fill in the table below with a ‘trace’ of the execution, i.e., all the variables’ values in all the iterations. Also include the potential values $s = x + y$, the ratios (as fractions) of potentials s_{i+1}/s_i for adjacent iterations, and Y/N indications of whether $s_{i+1}/s_i \leq 2/3$.

The entries labeled ‘input’, ‘division’, s , and s_{i+1}/s_i should be filled from top to bottom, corresponding to the recursive calls; the ‘recursive answer’ and ‘output’ entries will need to be filled from bottom to top, corresponding to the ‘post-processing’ (Line 7) of each recursive call’s results. The entries for a, b should match those of a', b' one row above. Put ‘-’ for any entries that are not defined due to the base case.

Solution:

i	input		division		rec ans		output		s	s_{i+1}/s_i	$\leq 2/3 ?$
	x	y	q	r	a'	b'	a	b			
0	376	203	1	173	75	-88	-88	163	579	376/579	Y
1	203	173	1	30	-13	75	75	-88	376	203/376	Y
2	173	30	5	23	10	-13	-13	75	203	53/203	Y
3	30	23	1	7	-3	10	10	-13	53	30/53	Y
4	23	7	3	2	1	-3	-3	10	30	9/30	Y
5	7	2	3	1	0	1	1	-3	9	3/9	Y
6	2	1	2	0	1	0	0	1	3	1/3	Y
7	1	0	-	-	-	-	1	0	1	-	-

Running the algorithm, we find that `EXTENDED_EUCLID(376, 203)` gives us $(g, a, b) =$

$(1, -88, 163)$, and we can verify that indeed $(-88) \cdot 376 + 163 \cdot 203 = 1$.

Grading notes:

+1 Row $i = 0$ correct

+1 Row $i = 1$ correct

+1 Row $i = 2$ correct

+1 Row $i = 3$ correct

+1 Row $i = 4$ correct

+1 Row $i = 5$ correct

+1 Row $i = 6$ correct

+1 Giving the Bezout coefficients a, b and verifying that they satisfy $ax + by = g$.